

■ JavaScript Type Coercion — Cheat Sheet

■ 1. typeof Results

Value	typeof Result
undefined	undefined
null	object
NaN	number
[]	object
{}	object
function(){}	function

■ 2. + Operator Rules

- If either operand is a string → string concatenation
- Otherwise → numeric addition
- Unary +x → converts x to number

■ 3. Type Conversion Table

Value	To Number	To String	To Boolean
undefined	NaN	undefined	false
null	0	null	false
true	1	true	true
false	0	false	false
[]	0	""	true
[1]	1	"1"	true
[1,2]	NaN	"1,2"	true
{}	NaN	[object Object]	true
NaN	NaN	NaN	false

■ 4. Common Expressions and Outputs

[] + [] → ""

[] + {} → '[object Object]'

{ } + [] → '[object Object]'

{ } + { } → NaN

({}+{}) → '[object Object][object Object]'

[2] + [3] → '23'

`[2,3] + [4,5] → '2,34,5'`

`[2] - [1] → 1`

`+[] → 0`

`+{} → NaN`

`null + 1 → 1`

`undefined + 1 → NaN`

■ 5. Equality Rules

Expression	Output	Reason
<code>0 == false</code>	true	both → 0
<code>0 === false</code>	false	type mismatch
<code>" == 0</code>	true	<code>"</code> → 0
<code>[] == 0</code>	true	<code>[]</code> → 0
<code>[0] == 0</code>	true	<code>'0'</code> → 0
<code>[1] == true</code>	true	<code>1 == 1</code>
<code>[2] == true</code>	false	<code>2 != 1</code>
<code>null == undefined</code>	true	special rule
<code>null == 0</code>	false	no coercion
<code>NaN == NaN</code>	false	NaN never equals itself

■ 6. Special Gotchas

- `typeof null === 'object'` (historical bug)
- NaN is a number but `NaN !== NaN`
- `{}` at start of line = block, not object
- `+{} → NaN`
- `[] → ""` (string) or `0` (number)
- `{}` → `'[object Object]'` (string) or `NaN` (number)